CURRICULUM TRACE REPORT

Code Analysis, Flowchart Matrix & Integrated Concept Tracer

Subject Scope: Python Processing Matrices (Outputs 11–20)
Environment Verification: PyCharm Professional Edition Workspace IDE Log
Date Generated: October 29, 2026

1. Code Block 11: validate_results

main.py [PythonProject3]

```
def validate_results(students, pass_mark, max_mark, grace_pool):
    results = {}
    for name in students:
        marks = students[name]
        updated_marks = {}
        pool_left = grace_pool
        failed_count = 0
        for sub in marks:
            score = marks[sub]
            if score < pass_mark:
                needed = pass_mark - score
                if pool_left >= needed:
                    pool_left -= needed
                    score = pass_mark
            if score > max_mark: score = max_mark
            if score < pass_mark: failed_count += 1
            updated_marks[sub] = score
        if failed_count == 0: status = "Pass"
        else: status = "Fail (" + str(failed_count) + " subjects)"
        results[name] = {"marks": updated_marks, "status": status}
    return results
```

RUN CONSOLE OUTPUT

```
C:\Users\Happy\PycharmProjects\PythonProject3\.venv\Scripts\python.exe main.py
11: {'John': {'marks': {'Math': 35, 'Science': 45, 'History': 78}, 'status':
'Pass'}, 'Mary': {'marks': {'Math': 28, 'Science': 35, 'History': 85}, 'status':
'Fail (1 subjects)'}}
```



Computational Thinking	Decomposes multi-student transcript evaluation into single subject loops, calculating delta metrics against passing thresholds.
Operators & Types	Uses comparison (<, >=), compound assignment (-=), and string concatenation (+).
Tuples & Unpacking	Uses direct key-based lookup (students[name]) instead of item tuple unpacking layouts.
Strings	Dynamically builds student status tokens via explicit integer string casting str(failed_count).
Code Analysis	Tracks local failure criteria; if the delta exceeds pool_left, the score remains un-adjusted and increments failure markers.
Algorithms & Pseudocode	Iterate students → reset local grace pool → process subjects → allocate pool if sufficient → count remaining failures → log structural record.
Pattern Generation & Loops	Implements a nested loop configuration where the outer handles collections and the inner evaluates sub-records.
Nested Lists & Tuples	

	Manages compound dictionary structures structured as Dict[str, Dict[str, int]] mirroring grade reports.

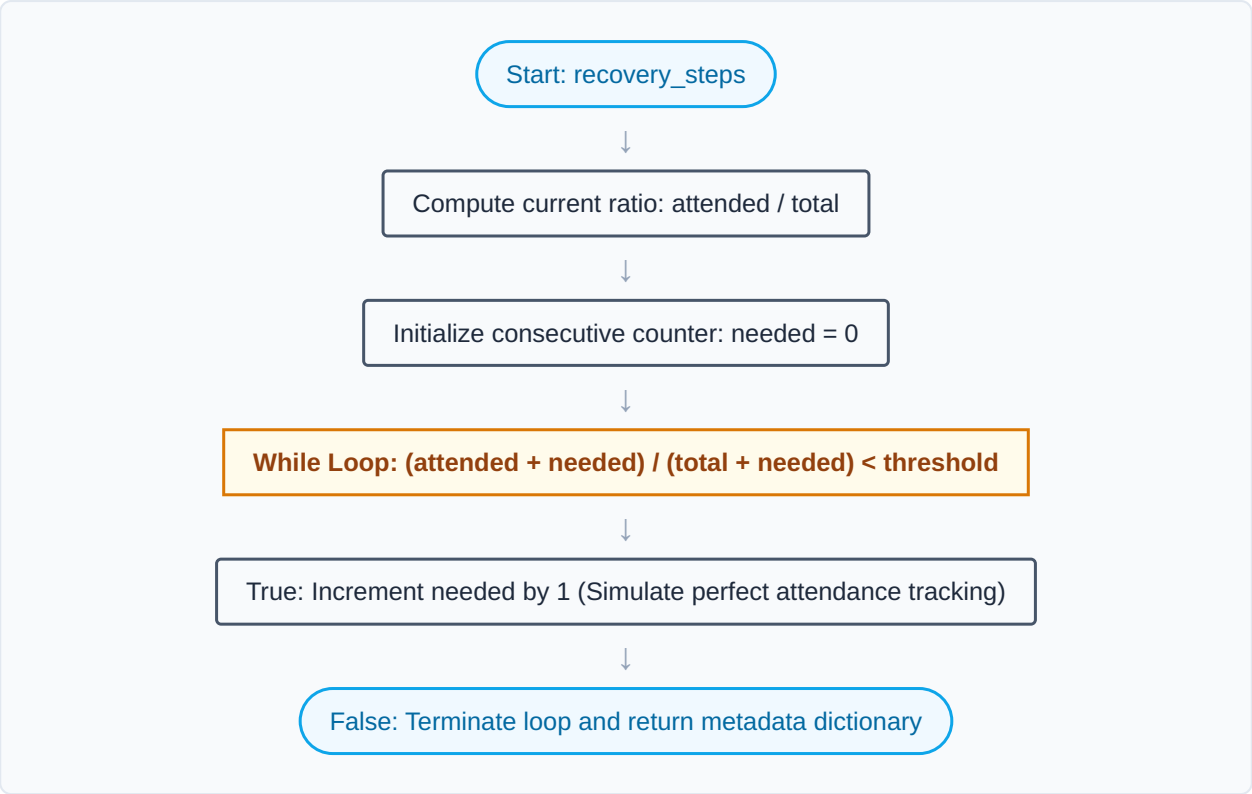
2. Code Block 12: recovery_steps

main.py [PythonProject3]

```python
def recovery_steps(attended, total, threshold=0.75):
    current = attended / total
    needed = 0
    while (attended + needed) / (total + needed) < threshold:
        needed += 1
    return {"current": current, "consecutive_needed": needed}
```

RUN CONSOLE OUTPUT

```
12: {'current': 0.6, 'consecutive_needed': 12}
```



Computational Thinking	Determines precision distance boundaries to converge a metric ratio toward target criteria.
Operators & Types	Implements standard true division (/) resolving to floating points alongside increment paths (+=).
Tuples & Unpacking	Evaluates scalar parameter types directly without requiring data container deconstruction.

Strings	Features fixed dictionary keys for metadata packing without variable format processing.
Code Analysis	Tracks convergence bounds where numerators and denominators scale uniformly until meeting target values.
Algorithms & Pseudocode	Evaluate initial division ratio → simulate extra perfect entries via continuous while evaluation loop → count iterations → return data block.
Pattern Generation & Loops	Employs a variable state while loop dependent on mathematical expression criteria updates.
Nested Lists & Tuples	Maintains a flat numerical single-tier configuration with zero nested arrays.

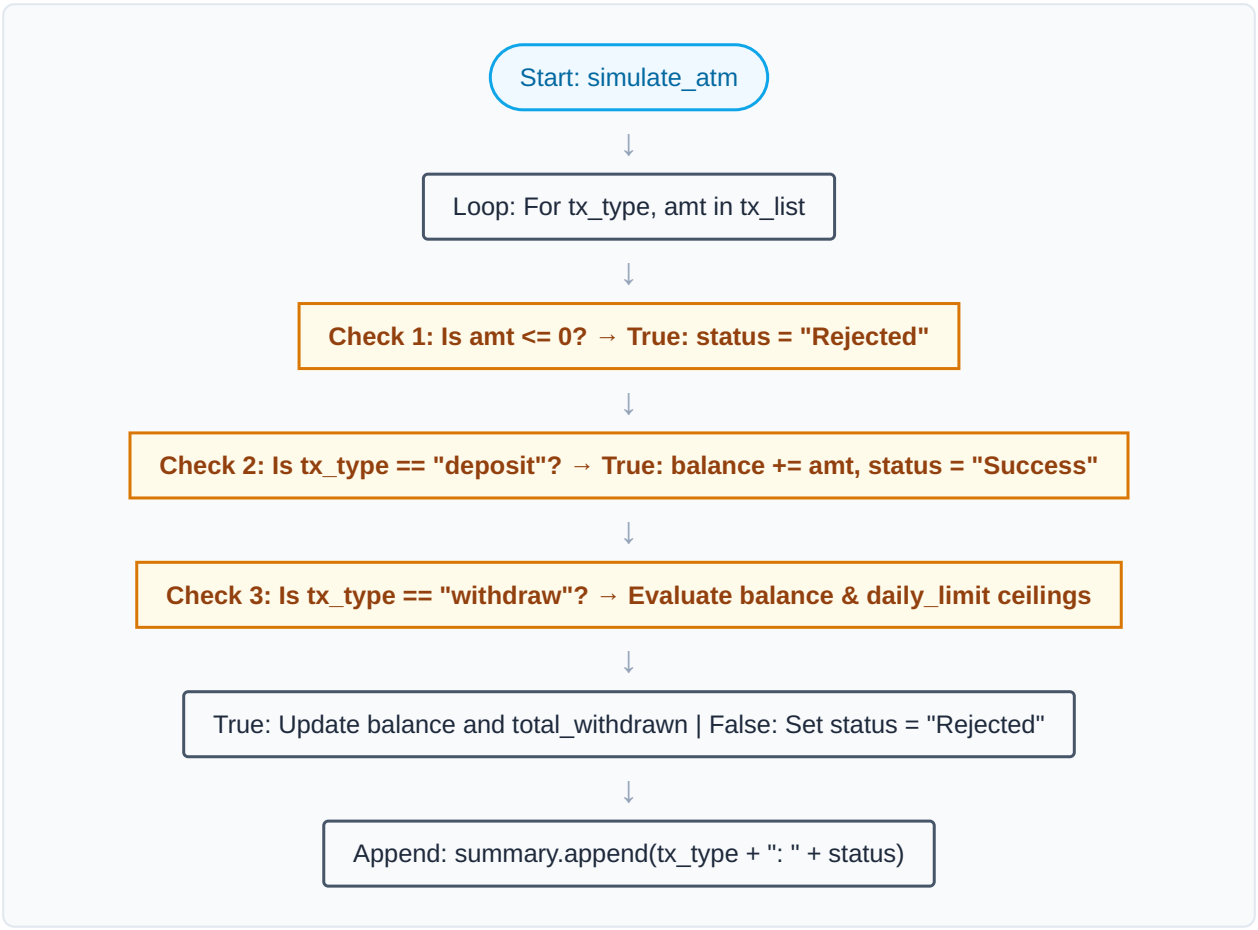
3. Code Block 13: simulate_atm

main.py [PythonProject3]

```
def simulate_atm(balance, tx_list, daily_limit=50000):
    total_withdrawn = 0
    summary = []
    for tx_type, amt in tx_list:
        if amt <= 0: status = "Rejected"
        elif tx_type == "deposit":
            balance += amt
            status = "Success"
        elif tx_type == "withdraw":
            if amt > balance or total_withdrawn + amt > daily_limit: status = "Re
            else:
                balance -= amt
                total_withdrawn += amt
                status = "Success"
        summary.append(tx_type + ": " + status)
    return {"balance": balance, "history": summary}
```

RUN CONSOLE OUTPUT

```
13: {'balance': 8000, 'history': ['withdraw: Success', 'deposit: Success']}
```



Computational Thinking	Models automated financial transactional ledgers by testing multi-conditional constraints against local balance scopes.
Operators & Types	Combines basic numeric comparisons with compound boolean evaluation markers (or).
Tuples & Unpacking	Unpacks items from list elements inline during the loop sequence structure (tx_type, amt).
Strings	Appends execution details using string combination concatenation metrics.
Code Analysis	Catches domain boundaries like negative value inputs immediately to protect account integrity.
Algorithms & Pseudocode	Iterate transactions → check boundary values → branch via type classification → apply limit safety filters → append log entries.
Pattern Generation & Loops	Executes linear iteration blocks across historical transactional data tracks.
Nested Lists & Tuples	Processes sequential structural sequences formatted as List[Tuple[str, int]].

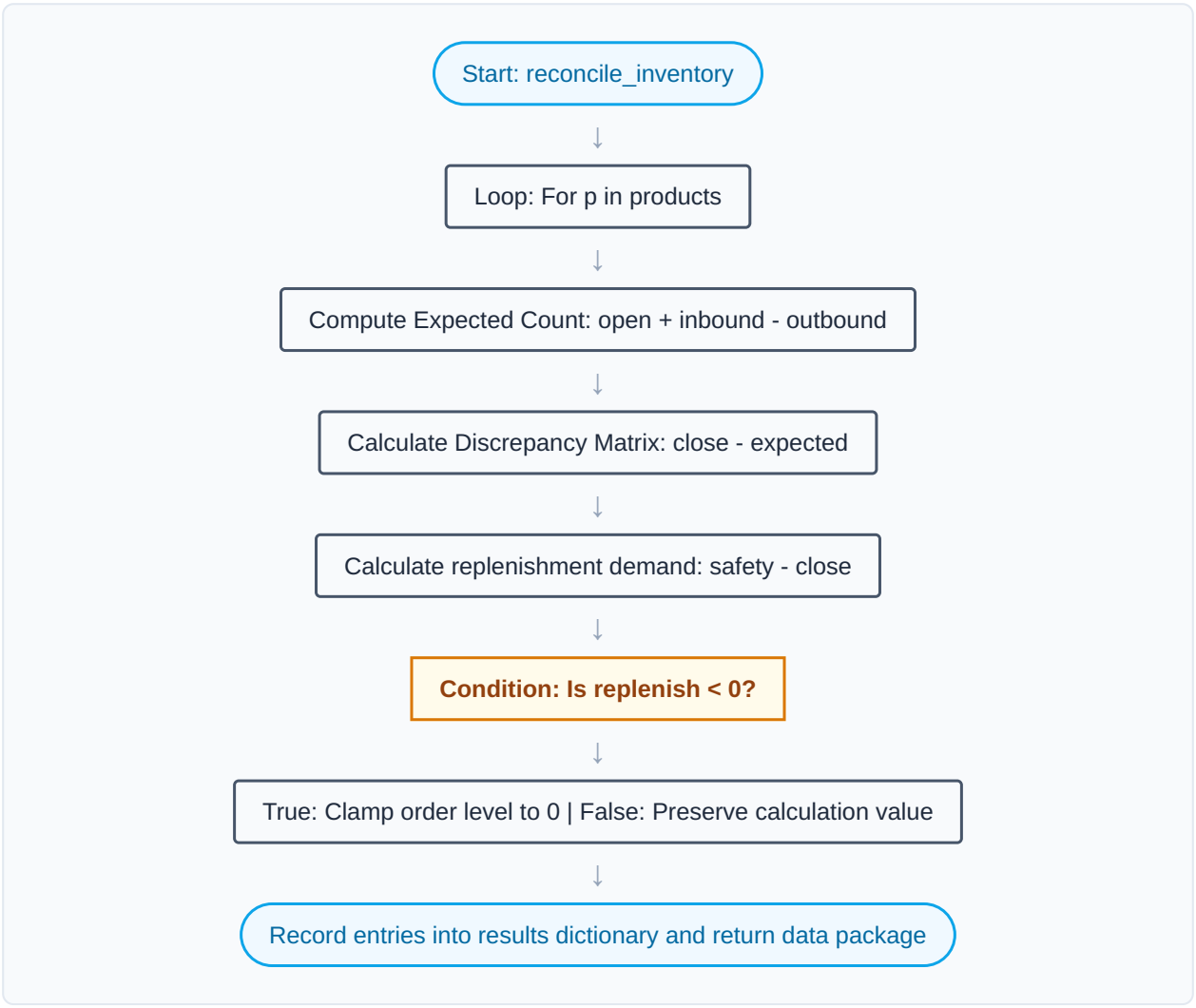
4. Code Block 14: reconcile_inventory

main.py [PythonProject3]

```
def reconcile_inventory(products):
    results = {}
    for p in products:
        v = products[p]
        expected = v["open"] + v["inbound"] - v["outbound"]
        discrepancy = v["close"] - expected
        replenish = v["safety"] - v["close"]
        if replenish < 0:
            replenish = 0
        results[p] = {"diff": discrepancy, "order": replenish}
    return results
```

RUN CONSOLE OUTPUT

```
14: {'ItemA': {'diff': -5, 'order': 15}}
```



Computational Thinking	Formulates audit reconciliation models by evaluating physical records against theoretical inventory updates.
Operators & Types	Uses basic arithmetic (+, -) combined with comparison operators for verification guards.
Tuples & Unpacking	Iterates through high-level object keys to scan sub-dictionary parameter configurations.
Strings	Employs literal token lookups matching internal nested data records.
Code Analysis	Includes floor limit clamping logic to prevent negative replenishment order numbers.
Algorithms & Pseudocode	Loop stock listings → calculate expected balance → compute record variance indicators → clamp safety quantities → save result.
Pattern Generation & Loops	Processes active elements via single-pass loop sequences.
Nested Lists & Tuples	Traverses multi-tiered nested maps structured as Dict[str, Dict[str, int]].

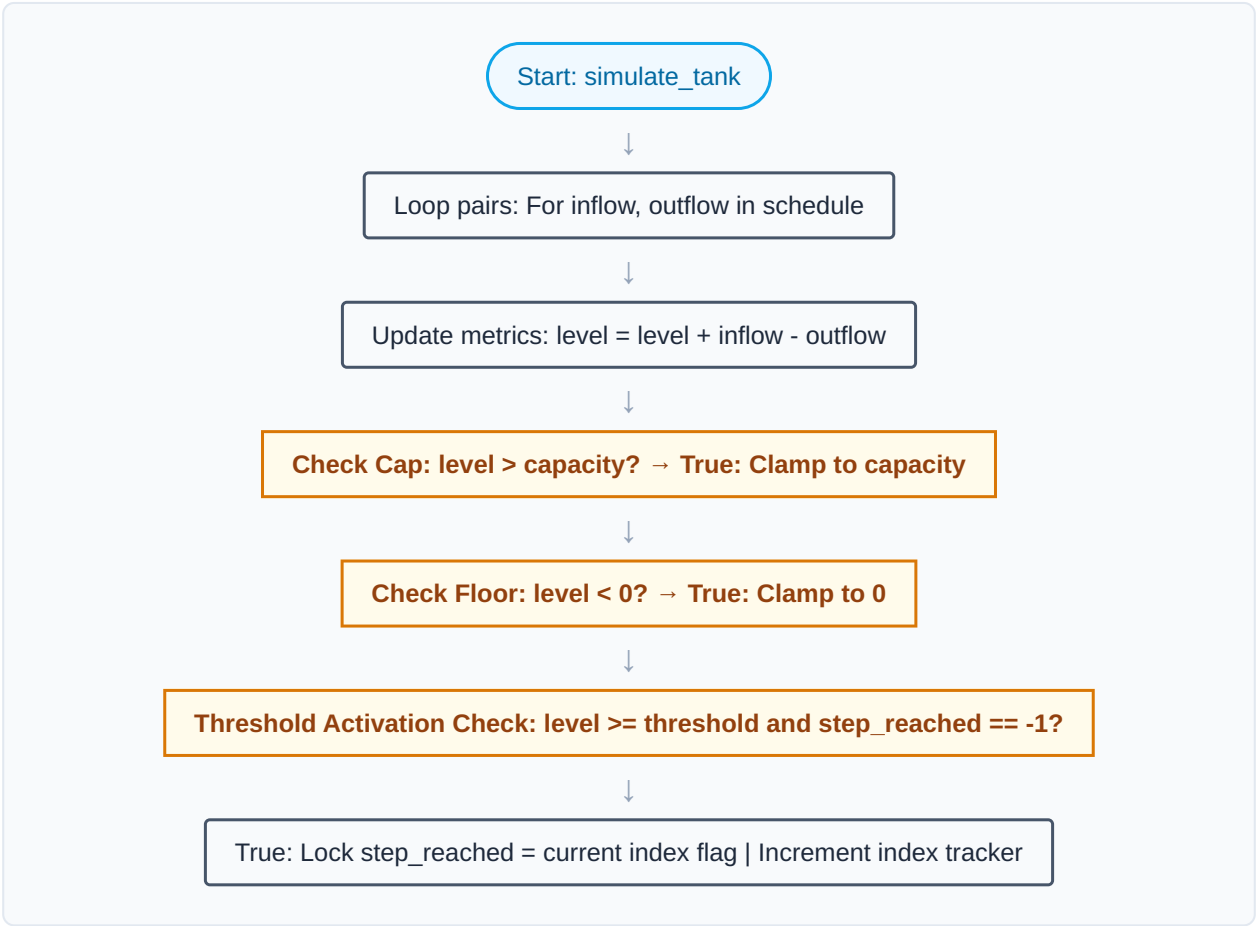
5. Code Block 16: simulate_tank

main.py [PythonProject3]

```
def simulate_tank(capacity, initial, schedule, threshold):
    level = initial
    step_reached = -1
    index = 1
    for inflow, outflow in schedule:
        level = level + inflow - outflow
        if level > capacity: level = capacity
        if level < 0: level = 0
        if level >= threshold and step_reached == -1:
            step_reached = index
        index += 1
    return {"final_level": level, "step": step_reached}
```

RUN CONSOLE OUTPUT

```
16: {'final_level': 850, 'step': 2}
```



Computational Thinking	Tracks environmental fluid models across operational steps while preserving threshold event logs.
Operators & Types	Combines numerical arithmetic with combined relational conditions (and).
Tuples & Unpacking	Uses direct unpacking methods to handle vector flow records (inflow, outflow).
Strings	Utilizes dictionary string tags for status extraction configurations.
Code Analysis	Enforces bounds clamping (0 to capacity) to capture overflow and drainage edge cases.
Algorithms & Pseudocode	Initialize levels → iterate delta schedules → calculate changes → run safety clamping checks → catch first threshold index → return status block.
Pattern Generation & Loops	Combines collection scanning with an index counter variable tracking execution steps.
Nested Lists & Tuples	Decodes sequential list containers consisting of paired coordinate markers (List[Tuple[int, int]]).

6. Code Block 18: find_best_route

main.py [PythonProject3]

```
def find_best_route(distance, fuel_price, vehicles):
    best_car = ""
    lowest_cost = 99999999
    for name in vehicles:
        v = vehicles[name]
        fuel_needed = (distance / v["mileage"]) + v["reserve"]
        cost = fuel_needed * fuel_price
        if cost < lowest_cost:
            lowest_cost = cost
            best_car = name
    return {"car": best_car, "cost": lowest_cost}
```

RUN CONSOLE OUTPUT

```
18: {'car': 'Sedan', 'cost': 2300.0}
```



Computational Thinking	Evaluates and standardizes cost variables across multiple objects to isolate optimal system efficiency.
Operators & Types	Blends floating-point division rules with linear comparison minimization routines.
Tuples & Unpacking	Iterates map keys to retrieve specific deep configuration attributes.
Strings	Maintains textual model identifiers for output categorization keys.
Code Analysis	Uses a high sentinel integer (99999999) to establish base parameters for optimization checks.
Algorithms & Pseudocode	Set optimization sentinel → loop vehicle metrics → calculate operational consumption indices → calculate cost totals → find minimum → return optimal record.
Pattern Generation & Loops	Executes clean single-pass loops across named entity blocks.
Nested Lists & Tuples	Navigates nested configuration models structured as Dict[str, Dict[str, float]].

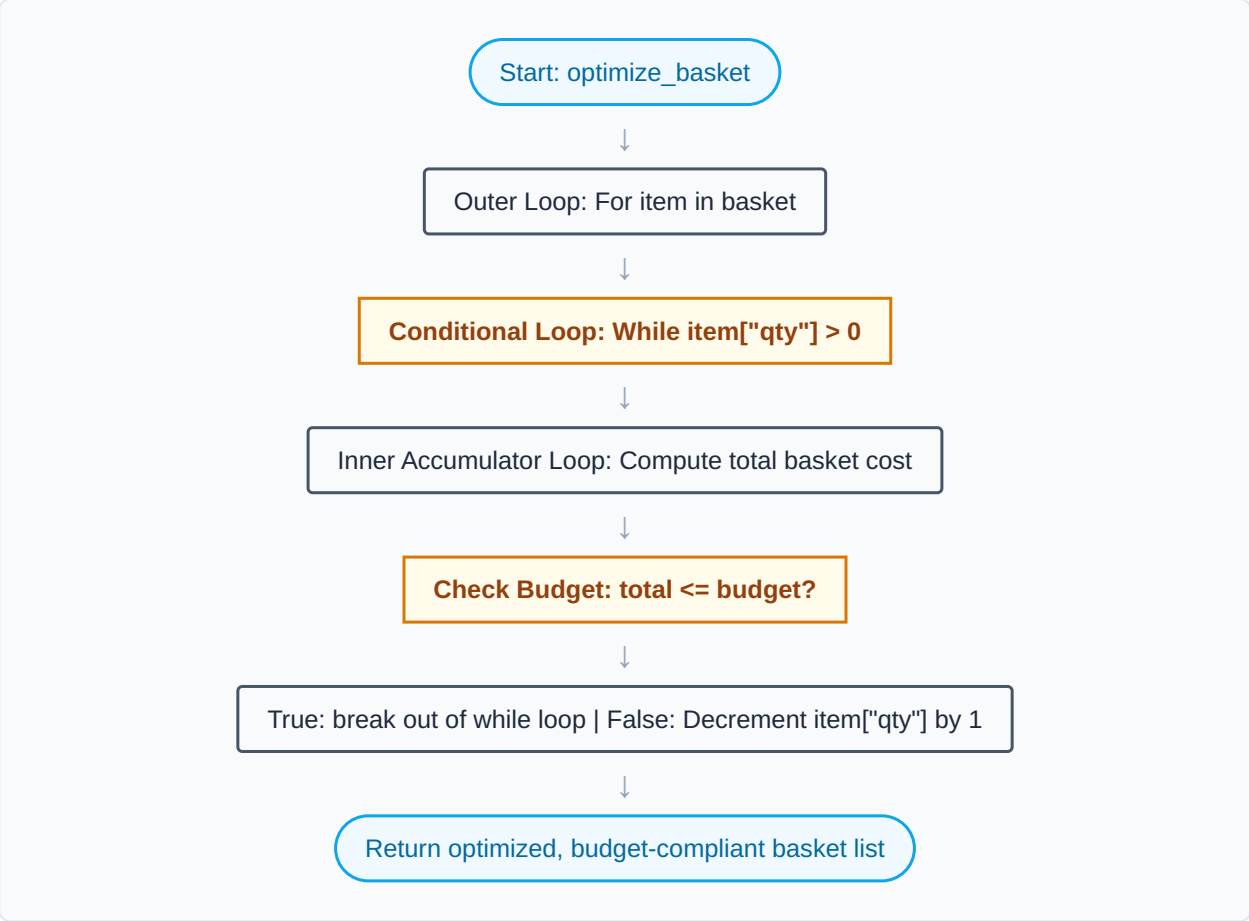
7. Code Block 19: optimize_basket

main.py [PythonProject3]

```
def optimize_basket(basket, budget):
    for item in basket:
        while item["qty"] > 0:
            total = 0
            for i in basket:
                total += i["qty"] * i["price"]
            if total <= budget:
                break
            item["qty"] -= 1
    return basket
```

RUN CONSOLE OUTPUT

```
19: [{'name': 'Shoes', 'price': 2000, 'qty': 1}, {'name': 'Shirt', 'price': 1000, 'qty': 2}]
```



Computational Thinking	

	Resolves budget ceilings by systematically scaling down item quantities within a resource-balancing matrix.
Operators & Types	Uses linear multiplication (*), compound subtraction (-=), and condition testing (<=).
Tuples & Unpacking	Loops across structured objects directly without multi-variable container extraction.
Strings	Accesses configuration values via constant structural keys.
Code Analysis	Features a nested calculation structure that repeatedly re-evaluates the total basket value after adjusting quantities.
Algorithms & Pseudocode	Loop items → enter conditional reduction loops → sum current total values → check budget compliance → break or decrement quantity → return list.
Pattern Generation & Loops	Implements a nested iteration matrix combining a for loop, a while loop, and an inner evaluation loop.
Nested Lists & Tuples	Processes structured collections organized as List[Dict[str, any]].

8. Code Block 20: university_admission

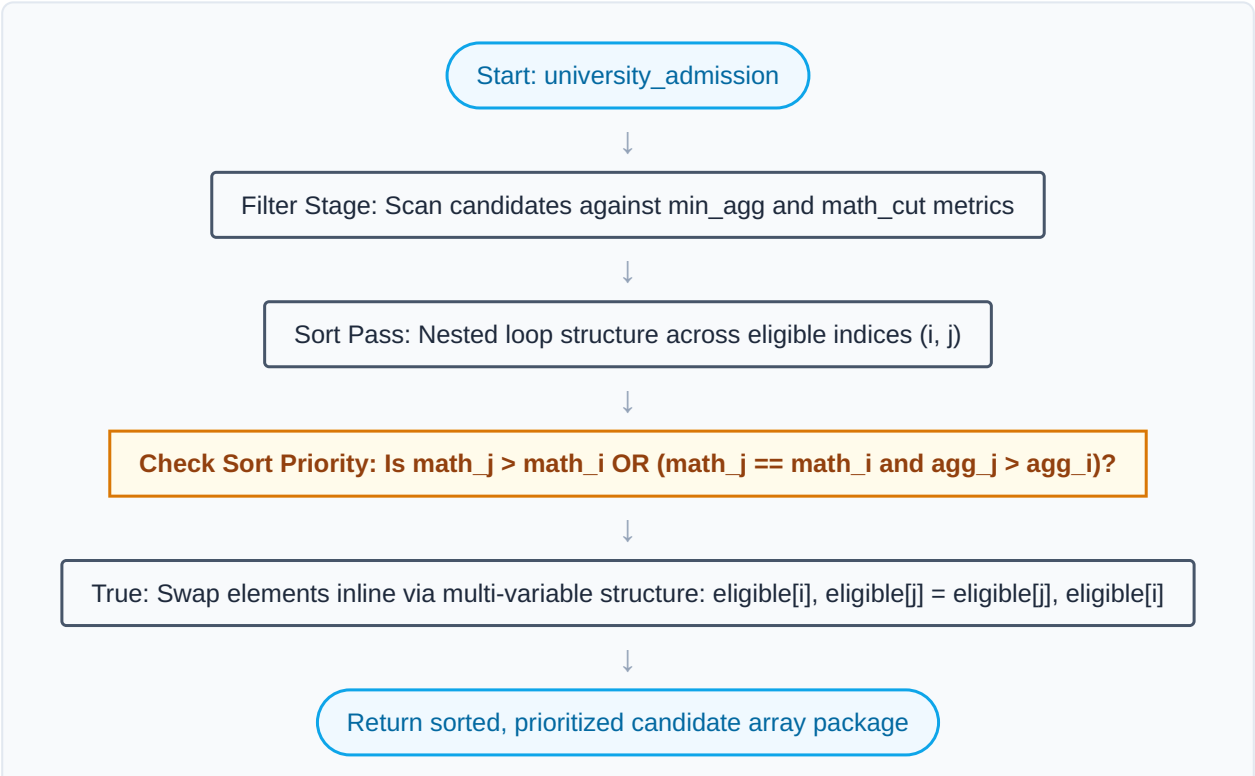
main.py [PythonProject3]

```python
def university_admission(candidates, min_agg, math_cut):
    eligible = []
    for c in candidates:
        if c["agg"] >= min_agg and c["math"] >= math_cut:
            eligible.append(c)

    for i in range(len(eligible)):
        for j in range(i + 1, len(eligible)):
            if (eligible[j]["math"] > eligible[i]["math"]) or (
                eligible[j]["math"] == eligible[i]["math"] and eligible[j]["agg"]
            ):
                eligible[i], eligible[j] = eligible[j], eligible[i]
    return eligible
```

RUN CONSOLE OUTPUT

```
20: [{'id': 2, 'agg': 88, 'math': 85}, {'id': 1, 'agg': 85, 'math': 90}]
```



Computational Thinking	Decouples candidate processing into an initial threshold filtering phase followed by a multi-attribute sorting phase.

Operators & Types	Combines index lookup expressions with compound relational criteria (and, or).
Tuples & Unpacking	Employs an inline variable assignment swap layout without intermediate storage variables.
Strings	References fixed map key strings to extract candidate performance values.
Code Analysis	Implements a tie-breaking sort strategy that prioritizes math scores first, then falls back to aggregate values.
Algorithms & Pseudocode	Filter records → build an eligibility list → run index-based sorting scans → execute swaps if order logic matches → return the sorted list.
Pattern Generation & Loops	Combines an initial single-pass loop with a nested sorting index iteration structure.
Nested Lists & Tuples	Manages arrays of structured dictionaries formatted as List[Dict[str, int]].